

Hardware-Info: Taster-Module im Vergleich

In diesem Projekt stehen zwei verschiedene Taster-Module zur Verfügung. Optisch dienen beide demselben Zweck (einen Tastendruck erfassen), **elektrisch funktionieren sie jedoch exakt gegensätzlich**.

Ein falsch konfigurierter Taster führt in ESPHome zu Dauer-Auslösungen ("Dauerfeuer") oder reagiert überhaupt nicht. Prüfen Sie genau, welches Modul Sie vorliegen haben, und passen Sie Ihren YAML-Code entsprechend an.



Modul A: M5Stack Unit Key (U144)

Dieses Modul kombiniert einen mechanischen Taster mit einer programmierbaren RGB-LED.

- **Elektrische Schaltung (Pull-Up):** Der Taster ist auf der Platine über einen Pull-Up-Widerstand mit der 3.3V-Spannung verbunden.
 - *Unbetätigt:* Das Signal liegt auf 3.3V (HIGH / logisch 1).
 - *Gedrückt:* Der Taster schließt den Stromkreis gegen Masse (GND). Das Signal fällt auf 0V (LOW / logisch 0).
- **Besonderheit:** Das Modul sendet die Taster- und LED-Signale über getrennte Leitungen. Beachten Sie die Farbcodierung der Jumper-Kabel (die gelbe und weiße Ader können im Vergleich zur Gehäusebeschriftung vertauscht sein!).

Die korrekte ESPHome-Konfiguration:

Weil der gedrückte Zustand ein LOW-Signal erzeugt, müssen wir die Logik im Code umdrehen (`inverted: true`), damit ESPHome den Druck als "AN" (ON) versteht.

```
binary_sensor:
  - platform: gpio
    pin:
      number: [Ihr GPIO Pin]
      mode: INPUT_PULLUP # Aktiviert internen Widerstand zur Sicherheit
      inverted: true      # Zwingend erforderlich!
    name: "M5Stack Unit Key"
```



Modul B: Seeed Studio Button v1.3 (Grove)

Dieses Modul ist ein reiner mechanischer Taster ohne integrierte LED.

- **Elektrische Schaltung (Pull-Down on Board):** Dieses Modul hat bereits einen **Pull-Down-Widerstand** fest auf der Platine verlötet. Dieser zieht das Signal standardmäßig auf Masse (GND).
 - *Unbetätigt:* Das Signal liegt auf 0V (LOW / logisch 0).
 - *Gedrückt:* Der Taster verbindet die Signalleitung mit der Versorgungsspannung (VCC/3.3V). Das Signal springt auf HIGH (logisch 1).
- **Besonderheit:** Da das Signal exakt dem logischen Verständnis entspricht (Strom da = AN, kein Strom = AUS), darf die Logik nicht umgedreht werden.

Die korrekte ESPHome-Konfiguration:

Da der Widerstand bereits auf der Platine verbaut ist, benötigt der ESP32 keinen internen Widerstand (INPUT reicht völlig aus).

`binary_sensor:`

```
- platform: gpio
  pin:
    number: [Ihr GPIO Pin]
    mode: INPUT          # Interner Pull-Up darf NICHT aktiv sein!
    inverted: false      # (kann weggelassen werden)
  name: "Seeed Studio Button"
```

Zusammenfassung für die Praxis

- **Unit Key:** `mode: INPUT_PULLUP` und `inverted: true`
- **Seeed Button:** `mode: INPUT` und `inverted: false`

Tipp: Nutzen Sie zum Testen das Live-Log von ESPHome. Wenn der Taster im Log sofort nach dem Start sekundlich "State ON" meldet, ohne dass Sie ihn berühren, haben Sie höchstwahrscheinlich die Parameter `mode` oder `inverted` falsch gesetzt!